PRUEBA DE PROGRAMACIÓN EN PYTHON

- **Nombre y apellidos:**
- **Fecha:**

Instrucciones

- Responde a todos los ejercicios.
- Puedes utilizar las instrucciones y estructuras trabajadas en clase.
- No puedes utilizar listas ni definir funciones propias.
- Se valorará la corrección del algoritmo y claridad del código.
- Si no consigues resolver un ejercicio completo, escribe las partes que sepas realizar.

Ejercicio 1. Análisis de código (2 puntos)

Observa el siguiente programa:

```
total = 0
for numero in range(2, 11, 2):
    if numero % 4 == 0:
        total = total + numero
    else:
        total = total + 1
    if total > 10:
        break
    print(numero, total)
print("Resultado:", total)
```

a) (1 punto) Completa la tabla indicando los valores de las variables al finalizar cada iteración que se ejecute. Indica también si se ejecuta la instrucción `print(numero, total)`.

Iteración	numero	total	¿Se ejecuta print(numero, total)?
Nº			
Nº			
Nº			
Nº			
Nº			
Nº			
Nº			

b) (0,5 puntos) Escribe exactamente qué muestra el programa por pantalla.

c) (0,5 puntos) Si se eliminara la instrucción break, ¿qué valor tendría total al finalizar el bucle? (0,5 puntos). Justifica brevemente tu respuesta.

Ejercicio 2. Cálculo del precio de una entrada (2 puntos)

Un parque de atracciones establece las siguientes tarifas:

Edad	Precio
Menores de 6 años	Gratis
De 6 a 17 años	12 €
De 18 a 64 años	20 €
65 años o más	10 €

Además, los visitantes que tengan un **carné de descuento** obtienen una reducción del 25 % sobre el precio que les corresponda.

Escribe un programa en Python que:

1. Solicite la edad del visitante.
2. Pregunte si tiene carné de descuento (`s` o `n`).
3. Calcule el precio de la entrada aplicando la tarifa correspondiente y, cuando proceda, el descuento.
4. Muestre el precio final con dos cifras decimales.

Ejemplo de ejecución:

```
Introduce tu edad: 16
¿Tienes carné de descuento? (s/n): s
Precio final: 9.00 €
```

Puedes suponer que la edad es un número entero no negativo y que la respuesta sobre el carné siempre será `s` o `n`.

Ejercicio 3. Control de un aparcamiento (3 puntos)

Un aparcamiento dispone de **20 plazas**. Al comenzar, está vacío.

Escribe un programa en Python que muestre repetidamente el siguiente menú:

```
--- APARCAMIENTO ---
1. Entra un vehículo
2. Sale un vehículo
3. Cerrar aparcamiento
```

El usuario seleccionará una opción y el programa actuará de la siguiente manera:

- **Opción 1:** Si quedan plazas libres, entra un vehículo y se actualiza la ocupación. Si está completo, muestra `Aparcamiento completo`.
- **Opción 2:** Si hay vehículos aparcados, sale uno y se actualiza la ocupación. Si está vacío, muestra `No hay vehículos`.
- **Opción 3:** El programa termina.
- **Cualquier otra opción:** Muestra `Opción no válida`.

Al cerrar el aparcamiento, el programa debe mostrar:

1. El número de vehículos que quedan aparcados.
2. El número total de vehículos que han entrado durante la ejecución.

Ejemplo de ejecución:

```
--- APARCAMIENTO ---
1. Entra un vehículo
2. Sale un vehículo
3. Cerrar aparcamiento

Opción: 1
Vehículo registrado. Ocupación: 1

Opción: 1
Vehículo registrado. Ocupación: 2

Opción: 2
Vehículo retirado. Ocupación: 1

Opción: 3
Aparcamiento cerrado.
Vehículos aparcados: 1
Total de vehículos que han entrado: 2
```

Condiciones:

- Utiliza un bucle para mantener el menú activo hasta seleccionar la opción 3.
- La ocupación nunca puede ser negativa ni superar las 20 plazas.
- No utilices listas ni funciones propias.
- Puedes suponer que el usuario introduce números enteros.

Ejercicio 4. La mayor racha de victorias (3 puntos)

Un jugador participa en varias partidas. En cada una puede ganar o perder. Queremos averiguar cuál ha sido su **mayor racha de victorias consecutivas**.

Escribe un programa en Python que:

1. Pregunte cuántas partidas se han jugado.
2. Solicite el resultado de cada partida:
    - `1` si ha ganado.

    - `0` si ha perdido.
3. Al finalizar, muestre:
    - El número total de victorias.
    - La mayor cantidad de victorias consecutivas que ha conseguido.

Ejemplo de ejecución:

```
¿Cuántas partidas? 8

Partida 1 (1 = victoria, 0 = derrota): 1
Partida 2 (1 = victoria, 0 = derrota): 1
Partida 3 (1 = victoria, 0 = derrota): 0
Partida 4 (1 = victoria, 0 = derrota): 1
Partida 5 (1 = victoria, 0 = derrota): 1
Partida 6 (1 = victoria, 0 = derrota): 1
Partida 7 (1 = victoria, 0 = derrota): 0
Partida 8 (1 = victoria, 0 = derrota): 1

Victorias totales: 6
Mayor racha de victorias: 3
```

Condiciones:

- Puedes suponer que se ha jugado al menos una partida.
- Los resultados introducidos siempre serán `0` o `1`.
- No puedes utilizar listas ni definir funciones propias.
- El programa debe funcionar también si el jugador pierde todas las partidas o las gana todas.